

Sistema Inteligente para Visualização e Replanejamento de Atividades

(SIVIRA)

Documentação do Módulo Factory

Módulo: `factory/`

Padrão de Design: Factory Pattern + Singleton

Sumário

1	Introdução	2
1.1	Arquivos do Módulo	2
2	Fábrica de Equipamentos	3
2.1	Visão Geral	3
2.2	Estrutura da Classe	3
2.3	Equipamentos Criados	3
2.3.1	Refrigeração e Congelamento	3
2.3.2	Cocção	3
2.3.3	Mistura e Preparação	3
2.3.4	Modelagem e Divisão	4
2.3.5	Fermentação e Armazenamento	4
2.3.6	Pesagem e Embalagem	4
2.3.7	Bancadas	4
2.4	Características Técnicas	4
2.4.1	Capacidades	4
2.4.2	Setores	4
2.5	Correções Implementadas	4
3	Fábrica de Funcionários	5
3.1	Visão Geral	5
3.2	Padrão Singleton	5
3.3	Características do Singleton	5
3.4	Métodos Principais	5
3.4.1	carregar_funcionarios()	5
3.4.2	obter_funcionario_por_id(id)	6
3.4.3	obter_funcionarios_por_setor(setor)	6
3.4.4	obter_funcionarios_por_profissao(profissao)	6
3.5	Estrutura do JSON	6
3.6	Campos do Funcionário	7
3.7	Tipos de Folga Suportados	7
3.8	Funcionários Cadastrados	8
3.9	Compatibilidade com Código Legado	8
4	Integração com o Sistema	9
4.1	Uso da Fábrica de Equipamentos	9
4.2	Uso da Fábrica de Funcionários	9
4.3	Benefícios do Padrão Singleton	9
5	Estatísticas	10
5.1	Resumo do Módulo	10
5.2	Distribuição por Arquivo	10
5.3	Equipamentos por Setor	10
6	Padrões de Design	11
6.1	Factory Pattern	11
6.2	Singleton Pattern	11
6.3	Separation of Concerns	11

7 Conclusão	12
7.1 Próximos Passos	12

Introdução

O módulo `factory/` implementa o padrão de design Factory Pattern para criação centralizada de objetos no sistema SIVIRA. Este módulo é responsável por:

- Criar instâncias de equipamentos com configurações padronizadas
- Gerenciar funcionários a partir de arquivos de configuração JSON
- Garantir consistência na criação de objetos através do padrão Singleton
- Facilitar a manutenção e evolução do sistema

Arquivos do Módulo

- `fabrica_equipamentos.py` (782 linhas): Factory para criação de equipamentos
- `fabrica_funcionarios.py` (342 linhas): Factory para gerenciamento de funcionários
- `__init__.py`: Módulo de inicialização

Fábrica de Equipamentos

Visão Geral

A classe `FabricaEquipamentos` é responsável pela criação padronizada de todos os equipamentos do sistema. Utiliza métodos estáticos para instanciar equipamentos com configurações específicas.

Estrutura da Classe

Classe `FabricaEquipamentos`

```
1 class FabricaEquipamentos:
2     """
3     Fabrica responsavel pela criacao dos equipamentos.
4     """
```

Equipamentos Criados

A fábrica cria 17 tipos diferentes de equipamentos, organizados por categoria:

2.3.1 Refrigeração e Congelamento

- `criar_camara_refrigerada_1()`: Câmara com temperatura 0-4°C, capacidade 200 caixas
- `criar_camara_refrigerada_2()`: Câmara com temperatura -18 a -7°C, capacidade 200 caixas
- `criar_freezer_1()`: Freezer com temperatura -25 a -10°C

2.3.2 Cocção

- `criar_fogao_1()`: Fogão de cozinha, 6 bocas, alta pressão
- `criar_fogao_2()`: Fogão de confeitaria, 4 bocas, baixa pressão
- `criar_forno_1()`: Forno turbo/lastro, 4 níveis, temperatura até 300°C
- `criar_forno_2()`: Forno combinado, 5 níveis, temperatura até 250°C
- `criar_fritadeira_1()`: Fritadeira de 15L com 2 frações

2.3.3 Mistura e Preparação

- `criar_batedeira_industrial_1()`: Capacidade 5-40kg, velocidade 100-500 RPM
- `criar_batedeira_planetaria_1()`: Capacidade 1-7kg, 3 velocidades
- `criar_masseira_1()`: Capacidade 15-50kg, mistura rápida/semi-rápida
- `criar_hot_mix_1()`: Capacidade 5-30kg com cocção integrada

2.3.4 Modelagem e Divisão

- **criar_divisora_de_massas_1()**: Capacidade 30-3000g, 30 divisões
- **criar_modeladora_de_paes_1()**: Modelagem de pães, capacidade 30-300g
- **criar_modeladora_de_salgados_1()**: Modelagem de salgados, capacidade 20-200g

2.3.5 Fermentação e Armazenamento

- **criar_armario_fermentador_1()**: Temperatura 30-40°C, 9 níveis
- **criar_armario_esqueleto_1()**: Armazenamento com 32 níveis

2.3.6 Pesagem e Embalagem

- **criar_balanca_digital_1()**: Pesagem 1-30000g, precisão 1g
- **criar_embaladora_1()**: Embalagem a vácuo, capacidade 100-5000g

2.3.7 Bancadas

- **criar_bancada_1()**: Bancada com 4 frações divisíveis

Características Técnicas

2.4.1 Capacidades

Cada equipamento possui capacidades mínimas e máximas bem definidas:

- **Misturadoras**: 1kg a 50kg dependendo do tipo
- **Fornos**: 4 a 5 níveis com temperaturas de 150°C a 300°C
- **Câmaras**: 200 caixas com controle de temperatura
- **Fritadeiras**: 15L com sistema de frações

2.4.2 Setores

Os equipamentos são distribuídos entre os setores:

- **ALMOXARIFADO**: Câmaras refrigeradas, freezer
- **COZINHA**: Fogão 1, fritadeira
- **CONFEITARIA**: Fogão 2, batedeiras, hot mix
- **PANIFICACAO**: Fornos, masseira, modeladoras, fermentador

Correções Implementadas

Durante o desenvolvimento, foram realizadas as seguintes correções:

- Capacidade de câmaras refrigeradas: 20 → 200 caixas
- Faixa de temperatura: ajustada para valores operacionais reais
- Número de operadores de fogões: ajustado para capacidade real
- Capacidades mínimas de bocas: reduzidas para permitir batches pequenos (2000g → 500g)

Fábrica de Funcionários

Visão Geral

A classe `FabricaFuncionarios` implementa o padrão Factory combinado com Singleton para gerenciar funcionários. Diferente da fábrica de equipamentos, esta carrega dados de um arquivo JSON, permitindo fácil manutenção sem modificar código.

Padrão Singleton

Implementação do Singleton

```
1 class FabricaFuncionarios:
2     """
3     Factory para criacao de funcionarios a partir de arquivo JSON.
4
5     Singleton: Garante que apenas uma instancia da fabrica existe,
6     mantendo os mesmos objetos Funcionario em memoria durante toda
7     a execucao.
8     """
9
10    _instance = None
11    _funcionarios_carregados = False
12
13    def __new__(cls, caminho_json: str = ...):
14        if cls._instance is None:
15            cls._instance = super(FabricaFuncionarios, cls).__new__(
16                cls)
17            return cls._instance
```

Características do Singleton

- **Instância única:** Garante apenas um objeto `FabricaFuncionarios` em memória
- **Estado preservado:** Funcionários mantêm ocupações entre múltiplas chamadas
- **Evita recarregamento:** Dados JSON carregados apenas uma vez
- **Consistência:** Mesmos objetos `Funcionario` em toda aplicação

Métodos Principais

3.4.1 `carregar_funcionarios()`

Carrega funcionários do arquivo JSON. Se já foram carregados anteriormente (singleton), retorna a lista existente preservando ocupações.

Método carregar_funcionarios

```
1 def carregar_funcionarios(self) -> List[Funcionario]:
2     """
3     Carrega funcionarios do arquivo JSON.
4
5     Singleton: Se ja foi carregado antes, retorna a mesma lista
6     de funcionarios (com ocupacoes preservadas).
7     """
8     if FabricaFuncionarios._funcionarios_carregados:
9         return self.funcionarios_disponiveis
10
11     # Carrega do JSON...
12     FabricaFuncionarios._funcionarios_carregados = True
13     return self.funcionarios_disponiveis
```

3.4.2 obter_funcionario_por_id(id)

Busca funcionário específico por ID.

3.4.3 obter_funcionarios_por_setor(setor)

Retorna lista de funcionários que trabalham em um setor específico.

3.4.4 obter_funcionarios_por_profissao(profissao)

Retorna lista de funcionários com uma profissão específica.

Estrutura do JSON

O arquivo data/funcionarios/funcionarios.json segue esta estrutura:

Estrutura JSON

```
1 {
2   "funcionarios": [
3     {
4       "id": 1,
5       "nome": "Funcionario 1",
6       "setores": ["PANIFICACAO", "CONFEITARIA"],
7       "tipos_profissionais": ["PADEIRO"],
8       "regras_folga": [
9         {
10          "tipo": "DIA_FIXO_SEMANA",
11          "dia_semana": "SEXTA"
12        }
13      ],
14       "ch_semanal": 44,
15       "horario_inicio": "08:00",
16       "horario_final": "18:00",
17       "horario_intervalo": {
18         "inicio": "11:00",
19         "duracao_minutos": 60
20       },
21       "fip": 2.0
22     }
23   ]
24 }
```

Campos do Funcionário

- **id**: Identificador único
- **nome**: Nome do funcionário
- **setores**: Lista de setores onde pode trabalhar
- **tipos_profissionais**: Lista de profissões/habilidades
- **regras_folga**: Padrões de folga (dia fixo, n-ésimo dia do mês, etc.)
- **ch_semanal**: Carga horária semanal
- **horario_inicio**: Horário de entrada
- **horario_final**: Horário de saída
- **horario_intervalo**: Início e duração do intervalo
- **fip**: Fator de Importância do Profissional

Tipos de Folga Suportados

- **DIA_FIXO_SEMANA**: Folga em dia fixo da semana (ex: toda segunda-feira)
- **DIA_FIXO_MES**: Folga em dia fixo do mês (ex: todo dia 15)
- **N_DIA_SEMANA_DO_MES**: Folga no n-ésimo dia da semana do mês (ex: segunda segunda-feira)

Funcionários Cadastrados

O sistema possui 9 funcionários cadastrados:

1. **Funcionário 1:** Padeiro (Panificação/Confeitaria) - FIP 2.0
2. **Funcionário 2:** Auxiliar de Padeiro (Panificação) - FIP 1.0
3. **Funcionário 3:** Auxiliar de Padeiro/Confeiteiro (Multi-setores) - FIP 3.0
4. **Funcionário 4:** Confeiteiro (Confeitaria) - FIP 2.0
5. **Funcionário 5:** Auxiliar de Confeiteiro (Confeitaria) - FIP 2.0
6. **Funcionário 6:** Auxiliar Multi-função (Confeitaria/Panificação) - FIP 1.0
7. **Funcionário 7:** Cozinheiro (Cozinha) - FIP 1.0
8. **Funcionário 8:** Almojarife (Almojarifado) - FIP 1.0
9. **Funcionário 9:** Almojarife (Almojarifado) - FIP 1.0

Compatibilidade com Código Legado

Para manter compatibilidade com código existente, a fábrica cria variáveis globais individuais:

```
1 funcionario_1 = funcionarios_disponiveis[0]
2 funcionario_2 = funcionarios_disponiveis[1]
3 # ... etc
```

Isso permite que código antigo continue funcionando enquanto novo código pode usar a API da fábrica.

Integração com o Sistema

Uso da Fábrica de Equipamentos

```
1 from factory.fabrica_equipamentos import FabricaEquipamentos
2
3 # Criar equipamento especifico
4 forno = FabricaEquipamentos.criar_forno_1()
5
6 # Acessar propriedades
7 print(forno.nome)
8 print(forno.capacidade_niveis_max)
```

Uso da Fábrica de Funcionários

```
1 from factory.fabrica_funcionarios import FabricaFuncionarios
2
3 # Criar instancia (singleton)
4 fabrica = FabricaFuncionarios()
5
6 # Carregar funcionarios
7 funcionarios = fabrica.carregar_funcionarios()
8
9 # Buscar por setor
10 panificacao = fabrica.obter_funcionarios_por_setor(
11     TipoSetor.PANIFICACAO
12 )
13
14 # Buscar por profissao
15 padeiros = fabrica.obter_funcionarios_por_profissao(
16     TipoProfissional.PADEIRO
17 )
```

Benefícios do Padrão Singleton

- **Memória eficiente:** Evita múltiplas instâncias desnecessárias
- **Estado consistente:** Ocupações de funcionários persistem durante execução
- **Performance:** JSON carregado apenas uma vez
- **Simplicidade:** Interface única e centralizada

Estatísticas

Resumo do Módulo

Métrica	Valor
Total de linhas de código	1.124
Arquivos Python	2
Equipamentos criáveis	17
Funcionários cadastrados	9
Setores suportados	4
Tipos de folga	3

Distribuição por Arquivo

Arquivo	Linhas	%
fabrica_equipamentos.py	782	69.6%
fabrica_funcionarios.py	342	30.4%
Total	1.124	100%

Equipamentos por Setor

Setor	Equipamentos
Panificação	7
Confeitaria	5
Cozinha	2
Almoxarifado	3
Total	17

Padrões de Design

Factory Pattern

Ambas as fábricas implementam o padrão Factory, que:

- Centraliza a lógica de criação de objetos
- Encapsula detalhes de implementação
- Facilita manutenção e evolução
- Permite substituição de implementações

Singleton Pattern

A FabricaFuncionarios adiciona o padrão Singleton para:

- Garantir instância única
- Preservar estado durante execução
- Evitar recarregamento desnecessário
- Manter consistência de dados

Separation of Concerns

O módulo separa claramente:

- **Configuração:** Dados JSON (funcionários)
- **Criação:** Métodos factory (equipamentos)
- **Modelo:** Classes de domínio (models/)
- **Negócio:** Lógica de alocação (services/)

Conclusão

O módulo `factory/` implementa uma arquitetura robusta e extensível para criação de objetos no sistema SIVIRA. A combinação dos padrões Factory e Singleton garante:

- **Consistência:** Objetos criados de forma padronizada
- **Manutenibilidade:** Configurações centralizadas e fáceis de modificar
- **Performance:** Singleton evita overhead de múltiplas instâncias
- **Flexibilidade:** JSON permite mudanças sem recompilar
- **Escalabilidade:** Fácil adicionar novos equipamentos ou funcionários

Próximos Passos

Possíveis melhorias futuras:

1. Migrar equipamentos para configuração JSON
2. Adicionar validação de schemas JSON
3. Implementar cache persistente entre execuções
4. Adicionar suporte a múltiplos perfis de configuração
5. Criar interface gráfica para edição de funcionários